



Facultad de Ingeniería
Maestría en Inteligencia Artificial (MAIA)
Aprendizaje por Refuerzo

Entrega final

Agente Q-learning tabular sobre MiniGrid-BlockedUnlockPickup-v0

Juan David Lara Camacho
Agustín Serrano

Universidad de los Andes
10 de mayo de 2026

Repositorio del proyecto
github.com/JuanLara18/minigrid-qlearning

Resumen

Se modela el ambiente **MiniGrid-BlockedUnlockPickup-v0** como un proceso de decisión de Markov determinista y se entrena un agente Q-learning tabular que lo resuelve consistentemente. La tarea exige encadenar cinco subtareas obligatorias bajo un único inventario: mover una bola que bloquea una puerta, recoger la llave, abrir la puerta, soltar la llave y recoger la caja objetivo. Para que el espacio de estados sea Markoviano sobre la función de recompensa con *shaping* por subtareas, definimos el estado como una tupla de ocho componentes que incluye banderas de progreso persistentes. Tras 10 000 episodios de entrenamiento el agente alcanza 100/100 de éxito en evaluación greedy con recompensa promedio +2.771 (máximo teórico +2.80), resolviendo el ambiente en 29 pasos.

1. Introducción

El aprendizaje por refuerzo modela un agente que interactúa con un ambiente por medio de acciones y aprende, a partir de la recompensa que recibe, una política que maximiza el retorno acumulado [1]. En el marco de los procesos de decisión de Markov (MDP) eso requiere definir tres objetos: el conjunto de estados $\mathcal{S}$, el conjunto de acciones $\mathcal{A}$ y la función de recompensa $R(s, a, s')$. Sobre esa formulación, Q-learning tabular [2] estima la función de valor estado-acción mediante actualizaciones incrementales basadas en la ecuación de Bellman.

Este trabajo aplica el marco a un ambiente con dependencias de orden explícitas entre subtareas: **MiniGrid-BlockedUnlockPickup-v0** de la librería MiniGrid [3]. La tarea no se puede resolver sin completar las subtareas en una secuencia fija, y la recompensa nativa del ambiente entrega señal positiva únicamente al completar el episodio. Eso hace necesario un diseño cuidadoso tanto del espacio de estados como de la función de recompensa para que Q-learning tabular converja en un número razonable de episodios.

El documento desarrolla en orden la formulación del ambiente como MDP (Sección 2), el algoritmo y los detalles de implementación relevantes (Sección 3), la configuración y observaciones del entrenamiento (Sección 4), la evaluación de la política aprendida (Sección 5), el proceso de desarrollo con las decisiones de diseño que llevaron a la formulación final (Sección 6) y las conclusiones generales del trabajo (Sección 7).

2. Ambiente y formulación MDP

El ambiente es una grilla rectangular con dos cuartos separados por un muro vertical conectados por una sola casilla con puerta. El agente parte en el cuarto izquierdo y debe llegar a una caja objetivo en el cuarto derecho. La puerta está cerrada y necesita una llave del mismo color. Una bola del mismo color que la puerta está justo enfrente, bloqueando el acceso. El agente carga un único objeto a la vez, así que para retirar la bola tiene que usar el inventario: **pickup** y **drop**. El ambiente es determinista una vez fijada la semilla.

2.1. Conjunto de estados

El estado se representa como una tupla de ocho componentes:

$$s = (\underbrace{(x, y)}_{\text{pos}}, \underbrace{d}_{\text{dir}}, \underbrace{c}_{\text{carrying}}, \underbrace{b}_{\text{ball_moved}}, \underbrace{k_p}_{\text{key_picked}}, \underbrace{o}_{\text{door_open}}, \underbrace{k_d}_{\text{key_dropped}}, \underbrace{x_p}_{\text{box_picked}}) \quad (1)$$

La Tabla 1 describe cada componente.

Componente	Tipo	Rango	Descripción
<code>pos</code>	$(\mathbb{Z}_{\geq 0})^2$	$(0..W-1, 0..H-1)$	posición del agente en la grilla
<code>dir</code>	$\mathbb{Z}$	$\{0, 1, 2, 3\}$	orientación: 0 E, 1 S, 2 W, 3 N
<code>carrying</code>	cadena	$\{\text{none}, \text{key}, \text{ball}, \text{box}\}$	objeto que el agente carga en el momento actual
<code>ball_moved</code>	booleano	$\{0, 1\}$	la bola ya no está en su posición inicial
<code>key_picked</code>	booleano	$\{0, 1\}$	el agente recogió la llave en algún momento del episodio
<code>door_open</code>	booleano	$\{0, 1\}$	la puerta está abierta
<code>key_dropped</code>	booleano	$\{0, 1\}$	el agente soltó la llave después de abrir la puerta
<code>box_picked</code>	booleano	$\{0, 1\}$	el agente cargó la caja al menos una vez

Tabla 1: Componentes del estado. Las cinco primeras son observables directamente del ambiente. Las tres últimas son banderas de progreso persistentes (una vez activan no vuelven a cero) y se incluyen para que la recompensa con *shaping* (Sección 2.3) sea Markoviana sobre (s, a, s') .

Las primeras cinco componentes son las directamente observables del ambiente: la posición y la orientación fijan qué celda mira el agente, y `carrying` junto con `ball_moved` y `door_open` describen el estado físico relevante para decidir la siguiente acción. Las tres últimas (`key_picked`, `key_dropped` y `box_picked`) parecen redundantes pero son indispensables para la función de recompensa: el *shaping* entrega cada bonificación una sola vez por episodio, así que sin estas banderas la misma transición (s, a, s') podría producir recompensas distintas según la historia previa, lo que rompería la propiedad Markoviana del modelo.

El tamaño teórico del espacio, tomando todos los productos cartesianos posibles, es $W \cdot H \cdot 4 \cdot 4 \cdot 2^5$, del orden de 10^4 para la grilla por defecto (aproximadamente 11×6). La cota es laxa: las restricciones de consistencia entre banderas (`key_dropped` = 1 implica `door_open` = 1 y `key_picked` = 1, y `box_picked` = 1 implica las tres anteriores) eliminan la mayoría de las combinaciones. El entrenamiento del agente visita 1421 estados distintos.

2.2. Conjunto de acciones

El conjunto de acciones es $\mathcal{A} = \{0, 1, 2, 3, 4, 5\}$ y corresponde a las seis acciones discretas que MiniGrid expone para este tipo de tareas [3]. La Tabla 2 indica, para cada acción, los estados en los que es aplicable y el efecto que produce. Una acción ejecutada en un estado donde no es aplicable se admite sin error pero deja $s' = s$; solo avanza el contador de pasos.

Idx	Nombre	Aplicable en	Efecto
0	<code>left</code>	todos los estados	$d \mapsto (d - 1) \bmod 4$
1	<code>right</code>	todos los estados	$d \mapsto (d + 1) \bmod 4$
2	<code>forward</code>	estados con celda al frente transitable (vacía o puerta abierta)	avanza una celda en dirección d
3	<code>pickup</code>	<code>carrying</code> = <code>none</code> y un objeto recogible al frente	<code>carrying</code> toma el tipo del objeto, la celda queda vacía
4	<code>drop</code>	<code>carrying</code> $\neq$ <code>none</code> y celda al frente vacía	deja el objeto en esa celda, <code>carrying</code> pasa a <code>none</code>
5	<code>toggle</code>	puerta al frente y <code>carrying</code> = <code>key</code>	abre la puerta: <code>door_open</code> pasa de 0 a 1

Tabla 2: Acciones disponibles. Las seis acciones quedan disponibles en todo el espacio de estados; el agente aprende cuándo cada una tiene efecto y cuándo no a través del costo por paso de la Sección 2.3.

2.3. Función de recompensa

La función $R(s, a, s')$ suma tres ingredientes: un costo por paso constante, las bonificaciones de *shaping* por subtareas y la recompensa terminal del ambiente:

$$R(s, a, s') = \text{step_cost} + \sum_{\ell \in \mathcal{L}} \beta_{\ell} \Delta_{\ell}(s, s') + r_{\text{goal}} \mathbb{I}[s' \text{ terminal}]. \quad (2)$$

El indicador $\Delta_{\ell}(s, s')$ vale 1 cuando la bandera de la subtaska ℓ pasa de 0 en s a 1 en s' , y 0 en cualquier otro caso. La Tabla 3 lista los componentes.

Componente	Valor	Condición
step_cost	−0.001	cada paso, sin excepción
β_{ball}	+0.20	$b : 0 \rightarrow 1$
$\beta_{\text{key_p}}$	+0.30	$k_p : 0 \rightarrow 1$
β_{door}	+0.50	$o : 0 \rightarrow 1$
$\beta_{\text{key_d}}$	+0.30	$k_d : 0 \rightarrow 1$ (requiere $o = 1$ y $k_p = 1$)
β_{box}	+0.50	$x_p : 0 \rightarrow 1$
r_{goal}	+1.00	s' terminal (caja entregada)

Tabla 3: Componentes de la recompensa. La suma máxima por episodio, alcanzada cuando el agente completa todas las subtareas exactamente una vez y resuelve el episodio en N pasos, vale $2.80 - 0.001 N$.

La motivación del *shaping*: la recompensa nativa del ambiente solo es positiva al recoger la caja y vale 0 en cualquier otro paso. Para Q-learning tabular eso es problemático: mientras el agente no logre por azar toda la cadena de subtareas en un mismo episodio, no recibe ninguna señal positiva y los Q-valores no se diferencian entre trayectorias. En pruebas iniciales con la recompensa cruda el agente entrenó 10 000 episodios sin encontrar la caja una sola vez. Las cinco bonificaciones intermedias marcan hitos verificables del estado y dan señal útil mucho antes de que aparezca una trayectoria exitosa completa por azar.

Los valores específicos buscan que las bonificaciones intermedias no dominen a la recompensa terminal y que el costo por paso no las anule. Con **max_steps**= 576, un costo de paso de −0.01 acumula −5.76 en un episodio largo, lo que borra la suma de los β_{ℓ} ; por eso lo bajamos a −0.001. Las bonificaciones se distribuyeron cualitativamente según la dificultad percibida de cada subtaska y se ajustaron empíricamente hasta lograr convergencia consistente.

3. Algoritmo

3.1. Q-learning tabular off-policy

El agente usa la regla canónica de Q-learning sobre una tabla indexada por la tupla del estado:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]. \quad (3)$$

La exploración usa una política ε -greedy con ε que decae geométricamente desde 1.0 hasta un piso 0.05.

3.2. Tres detalles que no son obvios

Tres decisiones de implementación fueron necesarias para que el agente convergiera. No son obvias a partir de la ecuación de actualización pero condicionan que el algoritmo aprenda una política

ejecutable.

Bootstrap solo en terminated, no en truncated. La ecuación 3 multiplica $\max_{a'} Q(s', a')$ por γ . En la implementación, ese bootstrap se anula cuando `done=True`. La pregunta es qué cuenta como `done`. El ambiente devuelve dos banderas: `terminated` (el episodio acabó porque se alcanzó un estado terminal real, en nuestro caso recoger la caja) y `truncated` (el episodio se cortó por `max_steps`). Tratar las dos como equivalentes hace que el agente aprenda a evitar los estados cercanos al límite de pasos aunque sean buenos, porque cree que la trayectoria simplemente termina ahí sin valor futuro. Solo `terminated` debe anular el bootstrap.

Argmax con desempate aleatorio. La política greedy elige $a^* = \arg \max_a Q(s, a)$. Cuando varias acciones empatan en Q-valor, lo cual es frecuente al inicio del entrenamiento y en estados poco visitados, una implementación ingenua que devuelve siempre el primer índice produce ciclos deterministas en la política greedy: el agente queda atrapado en una secuencia periódica que no explora. El desempate aleatorio entre los índices con valor máximo rompe esos ciclos sin sesgar el aprendizaje en estados con un máximo único.

Shaping *first-time-only* dentro del estado. Las bonificaciones de subtarea se entregan una sola vez por episodio. Una implementación natural mantiene un registro externo (un diccionario de logros) y consulta ese registro al calcular la recompensa. El problema es que entonces la misma transición (s, a, s') puede devolver recompensas distintas según la historia previa del episodio, lo que rompe la propiedad Markoviana. Las banderas k_p, k_d, x_p dentro del estado resuelven esto: el indicador $\Delta_\ell(s, s')$ que decide si se entrega la bonificación se construye exclusivamente a partir de s y s' . El precio que se paga es un espacio de estados más grande (Sección 6.4).

3.3. Hiperparámetros

Parámetro	Valor	Justificación
alpha	0.1	Tasa de aprendizaje estándar para ambientes deterministas
gamma	0.99	Descuento alto: las subtareas están a decenas de pasos del retorno terminal
epsilon	1.0 $\rightarrow$ 0.05	Exploración total al inicio, mínimo residual de 5 % para no quedar pegados a un óptimo local
epsilon_decay	0.9995	Decaimiento geométrico lento: ϵ toca el piso alrededor del episodio 6 000
N_EPISODES	10 000	Suficiente para convergencia estable con margen
MAX_STEPS	576	Coincide con el límite interno del ambiente MiniGrid
env_seed	0	Fija la configuración inicial del ambiente
random_seed	42	Fija el desempate del argmax y la exploración del agente

Tabla 4: Hiperparámetros del entrenamiento. Las dos semillas hacen el experimento exactamente reproducible.

4. Entrenamiento

Se entrenó al agente durante 10 000 episodios con la configuración de la Sección 3. La Tabla 5 reporta la recompensa promedio sobre los 500 episodios previos a cada punto de control. La Figura 1 muestra la curva completa.

Episodio	ϵ	Recompensa promedio	Estados visitados
500	0.779	+0.201	1 258
1000	0.606	+1.300	1 383
1500	0.472	+1.812	1 383
2000	0.368	+1.531	1 386
2500	0.286	+1.737	1 387
3000	0.223	+1.651	1 416
3500	0.174	+1.477	1 416
4000	0.135	+1.300	1 416
5000	0.082	+1.118	1 421
7500	0.050	+2.464	1 421
10000	0.050	+2.722	1 421

Tabla 5: Progresión del entrenamiento: recompensa promedio sobre los últimos 500 episodios, valor de ϵ y cantidad acumulada de estados distintos visitados. La curva no es monótona; el texto la describe.

La curva tiene tres regiones cualitativas:

Fase 1: subida inicial (ep 1–1500).

ϵ cae de 1.0 a 0.5. La recompensa promedio sube de +0.20 a +1.81 porque el agente, todavía explorando fuerte, cae en las dos primeras subtareas (mover la bola, recoger la llave) con frecuencia creciente. Casi todo el espacio de estados útil aparece aquí: 1 383 de los 1 421 estados totales se ven antes del episodio 1 500.

Fase 2: plateau volátil (ep 1500–5500).

ϵ cae de 0.5 a ~ 0.07 pero la recompensa no sube: oscila entre +1.1 y +1.8, incluso retrocede (mínimo +1.118 en el episodio 5 000). El agente domina las primeras subtareas pero no encuentra de forma consistente la secuencia completa hasta la caja. El ϵ residual sigue siendo suficiente para que el ruido domine sobre la mejora.

Fase 3: convergencia (ep 5500–10000).

ϵ ya está en su piso de 0.05. La recompensa sube de +1.1 a +2.72. El agente ya tiene Q-valores estables para los estados raros del cuarto derecho (tras abrir la puerta) y la cadena completa se ejecuta consistentemente.

5. Evaluación

5.1. Evaluación greedy

Tras los 10 000 episodios de entrenamiento, se evalúa la política greedy sobre 100 episodios consecutivos en el mismo ambiente con la misma semilla. La Tabla 6 resume los números. La Figura 2 muestra seis fotogramas equiespaciados de un episodio greedy.

5.2. Robustez frente a variaciones de la evaluación

Para verificar que el resultado no depende del desempate específico del argmax, se evaluó la misma Q-tabla con cinco semillas distintas para el generador de números aleatorios del agente y con tres niveles de exploración residual durante la evaluación. La Tabla 7 resume los resultados sobre 100 episodios cada uno.

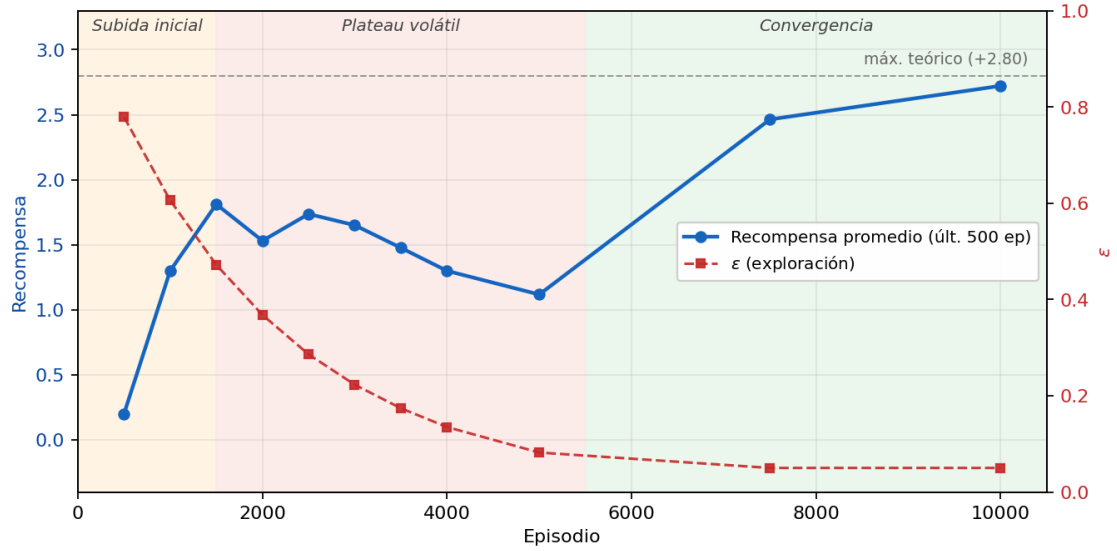


Figura 1: Curva de aprendizaje completa. La línea azul fina es la recompensa por episodio; la azul gruesa es su media móvil sobre 500 episodios. La línea punteada gris marca el máximo teórico +2.80. La línea roja punteada (eje derecho) es ϵ . El agente atraviesa una fase de *plateau* volátil entre los episodios 1 500 y 5 500 antes de subir a la zona del máximo teórico.

Métrica	Valor
Tasa de éxito	100 / 100
Recompensa promedio por episodio	+2.771
Pasos promedio por episodio	29.0
Máximo teórico de recompensa	+2.80
Pasos óptimos teóricos (cota inferior)	≥ 24

Tabla 6: Resultados de la evaluación greedy. La diferencia entre +2.771 y el máximo +2.80 se explica por el costo acumulado de los pasos: $29 \times (-0.001) = -0.029$.

6. Proceso de desarrollo

La formulación que finalmente convergió no fue la primera que probamos. El proceso pasó por tres versiones identificables. Esta sección las documenta porque las decisiones que se tomaron en cada iteración forman parte del razonamiento detrás de la solución final.

6.1. Versión inicial: no aprende

La primera implementación, antes del refinamiento del estado y del shaping, presentó tasa de éxito greedy de 0/100 y recompensa promedio -5.76 . El análisis identificó cinco problemas. La Tabla 8 los resume con la solución que se aplicó a cada uno.

6.2. Versión intermedia: aprende pero no es estable

Aplicados los fixes de estado, costo, bootstrap y desempate, el agente empezó a aprender durante el entrenamiento (avg reward $\approx +1.9$ en los últimos 500 episodios) pero la evaluación greedy seguía dando 0/100. La política aprendida no era ejecutable sin la exploración residual: cuando se quitaba ϵ , el agente quedaba atrapado en estados del cuarto derecho que no había visitado lo

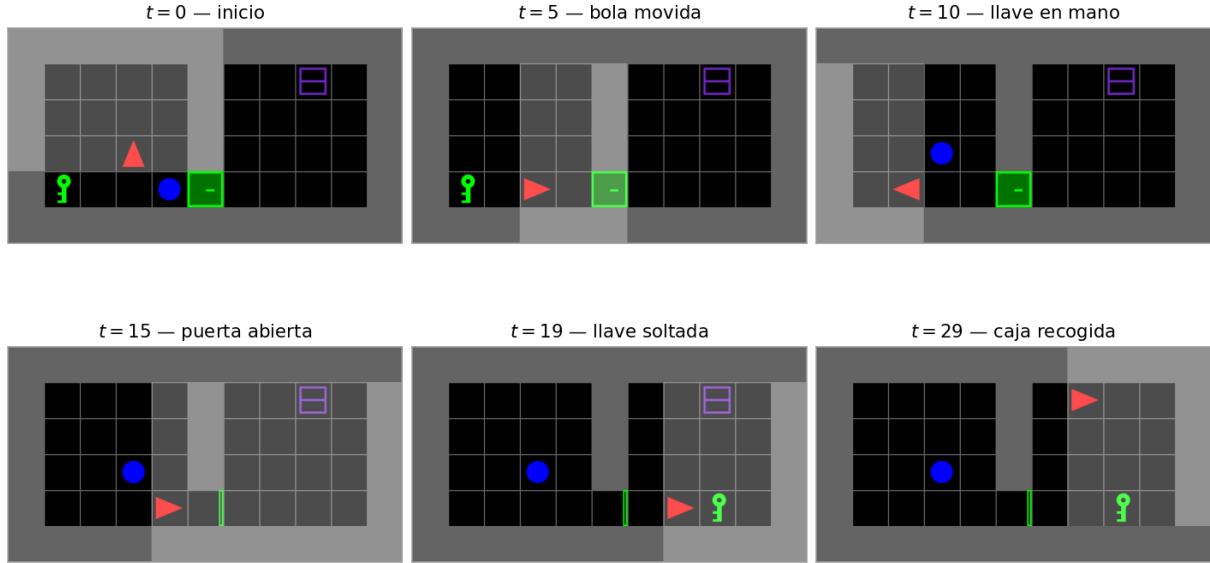


Figura 2: Seis fotografías equiespaciadas de un episodio greedy del agente entrenado. Fila superior: fase en el cuarto izquierdo (despeje de la bola, captura de la llave, acercamiento a la puerta). Fila inferior: cruce de la puerta, navegación en el cuarto derecho y recolección de la caja, que cierra el episodio.

suficiente para que la Q-tabla los distinguiera.

6.3. Versión final

Los bonus adicionales para las subtarefas finales (soltar la llave, recoger la caja) cerraron la brecha. La política greedy se volvió ejecutable y estable: 100/100 éxito, recompensa promedio +2.78. La Tabla 9 compara las tres iteraciones.

6.4. Un compromiso adicional: tamaño del estado

Hubo además una decisión técnica que se tomó al pulir la solución final, motivada por el principio Markoviano. Probamos dos formulaciones del estado:

Formulación compacta (5 componentes) (pos, d, c, b, o). Las banderas `key_picked`, `key_dropped` y `box_picked` no son parte del estado; viven en un registro externo que el ambiente consulta al construir la recompensa. Espacio visitado: 635 estados. Política greedy: 100/100, +2.776, 24 pasos.

Formulación markoviana (8 componentes) Como en la Sección 2: las tres banderas persistentes se incluyen en la tupla del estado. La recompensa $R(s, a, s')$ queda determinada exclusivamente por s, s' sin consultar historia externa. Espacio visitado: 1 421 estados. Política greedy: 100/100, +2.771, 29 pasos.

El compromiso es claro. La formulación compacta entrena ligeramente más rápido y converge a una trayectoria más corta, pero la recompensa no es Markoviana sobre (s, a, s') en el sentido estricto: la misma transición puede dar recompensa distinta según el historial. La formulación de 8 componentes paga un factor $\approx 2,2$ en tamaño del espacio y unos pocos pasos extra en la trayectoria, a cambio de cumplir la propiedad Markoviana que justifica el algoritmo de Q-learning. Optamos por la formulación de 8 componentes porque, en un proyecto cuyo objetivo es ilustrar el marco MDP, conservar la formalidad pesa más que ganar cinco pasos.

Estrategia	Éxito	Reward	Pasos
Greedy, seed=42	100 / 100	+2.771	29.0
Greedy, seed=0	100 / 100	+2.771	29.0
Greedy, seed=1	100 / 100	+2.771	29.0
Greedy, seed=7	100 / 100	+2.771	29.0
Greedy, seed=123	100 / 100	+2.771	29.0
ϵ -greedy, $\epsilon = 0.01$	99 / 100	+2.750	34.8
ϵ -greedy, $\epsilon = 0.05$	97 / 100	+2.696	49.4
ϵ -greedy, $\epsilon = 0.10$	93 / 100	+2.612	75.3

Tabla 7: Robustez. Greedy puro produce el mismo resultado bajo cinco semillas distintas, evidencia de que no hay empates en los estados visitados durante la trayectoria óptima. La degradación al introducir ϵ residual es gradual y simétrica, sin colapsos abruptos.

7. Conclusiones

- Q-learning tabular resuelve completamente el ambiente cuando se diseña con cuidado el estado y la recompensa. Con 1 421 estados distintos y seis acciones, la Q-tabla cabe holgadamente en memoria y converge en pocos minutos. No hizo falta función de aproximación.
- La representación del estado es la decisión más importante. Pasar de un flag sticky `has_key` a un `carrying_type` explícito fue lo que separó “no aprende” de “aprende”; incluir las tres banderas persistentes fue lo que cerró la propiedad Markoviana.
- El *shaping* de recompensa funciona si las bonificaciones marcan hitos verificables del estado (banderas que se levantan una sola vez), no acciones (`pickup`). Acciones son ambiguas; estados son no ambiguos.
- Algunos detalles de implementación tienen impacto outsized en el aprendizaje: bootstrap correcto (sólo en `terminated`) y desempate aleatorio en el argmax fueron necesarios para que la política greedy fuera ejecutable.
- El laberinto 8×7 (Sección A) sirvió como referencia verificable: BFS daba la respuesta exacta y se pudo confirmar que el agente la igualaba. Funcionó como sanity check en momentos donde no estaba claro si el problema era el algoritmo o la formulación.

8. Entregables y repositorio

La entrega final incluye, además de este documento, los siguientes artefactos: la Q-tabla entrenada (1 416 estados con sus Q-valores serializados), el código completo del ambiente y del agente con sus métodos de persistencia, un video del agente resolviendo el ambiente en 29 pasos, y las curvas de aprendizaje. Todo está disponible públicamente en el repositorio del proyecto:

github.com/JuanLara18/minigrid-qlearning

El README del repositorio detalla la estructura del código, las instrucciones de instalación y los comandos para reproducir el entrenamiento, la evaluación y la generación de los artefactos.

Problema	Causa	Solución aplicada
Estado ambiguo	Una bandera <code>has_key sticky</code> que no distinguía entre llevar la llave <i>ahora</i> y haberla llevado en algún momento. Los estados “cargó la bola” y “ya solté la bola” colapsaban.	Reemplazo por <code>carrying_type</code> (cadena con el tipo de objeto actual), más banderas auxiliares de progreso.
Costo por paso dominante	$-0.01 \times 576 = -5.76$, magnitud comparable a la suma de los bonus de subtarea.	Reducido a -0.001 , de modo que un episodio de 576 pasos cueste -0.576 y los bonus no queden anulados.
Bootstrap incorrecto	<code>done = terminated or truncated</code> . El bootstrap se anulaba al truncar por tiempo, lo que castigaba indebidamente a las trayectorias largas.	<code>done</code> usa solo <code>terminated</code> (Sección 3).
Ciclos en greedy	<code>argmax</code> devolvía siempre el primer índice empatado, lo que producía secuencias periódicas en estados sin política aún formada.	Desempate aleatorio entre los índices con valor máximo.
Shaping incompleto	No había señal positiva entre abrir la puerta y recoger la caja. El agenteabría la puerta y luego deambulaba sin saber qué hacer.	Bonus adicionales para soltar la llave (+0.30) y recoger la caja (+0.50).

Tabla 8: Los cinco problemas de la versión inicial y la solución aplicada en cada caso. Cada problema se diagnosticó por separado a partir de inspeccionar trayectorias y curvas de aprendizaje.

Versión	Reward train (últ. 500)	Éxito greedy	Reward greedy
Inicial	≈ -5.0	0/100	-5.76
Intermedia	$\approx +1.9$	0/100	$+0.42$
Final	$\approx +2.7$	100/100	$+2.78$

Tabla 9: Evolución entre versiones. La métrica más informativa es la columna “Reward greedy”: entrena bien no implica ejecuta bien.

A. Trabajo adicional: laberinto rectangular 8×7

En paralelo al ambiente principal trabajamos un laberinto rectangular 8×7 con muros internos. Lo incluimos como apéndice porque la solución óptima exacta se obtiene con BFS, lo que dio un punto de referencia objetivo para validar la implementación de Q-learning durante el desarrollo.

El ambiente es una grilla de 8 filas por 7 columnas (56 celdas) con 37 muros internos. El agente parte en (6,0) y debe alcanzar (1,6). El estado es $s = (x, y)$; las acciones son $\mathcal{A} = \{N, S, E, W\}$ siempre aplicables; la recompensa es +100 al alcanzar la meta y -1 en cualquier otro paso. Con Q-learning tabular ($\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon : 1.0 \rightarrow 0.05$ decay 0.999, 2000 episodios) el agente alcanza 100/100 de éxito greedy, retorno +76 y 25 pasos por episodio, idéntico al camino óptimo BFS sobre el grafo (55 de 56 estados visitados durante el entrenamiento).

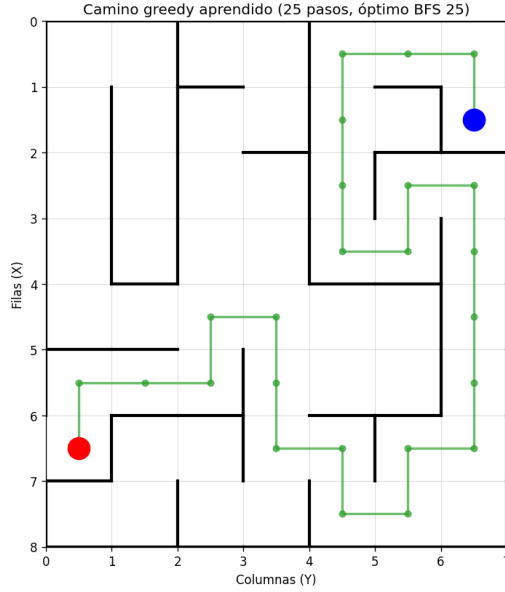


Figura 3: Laberinto y trayectoria greedy aprendida. Coincide con el óptimo BFS.

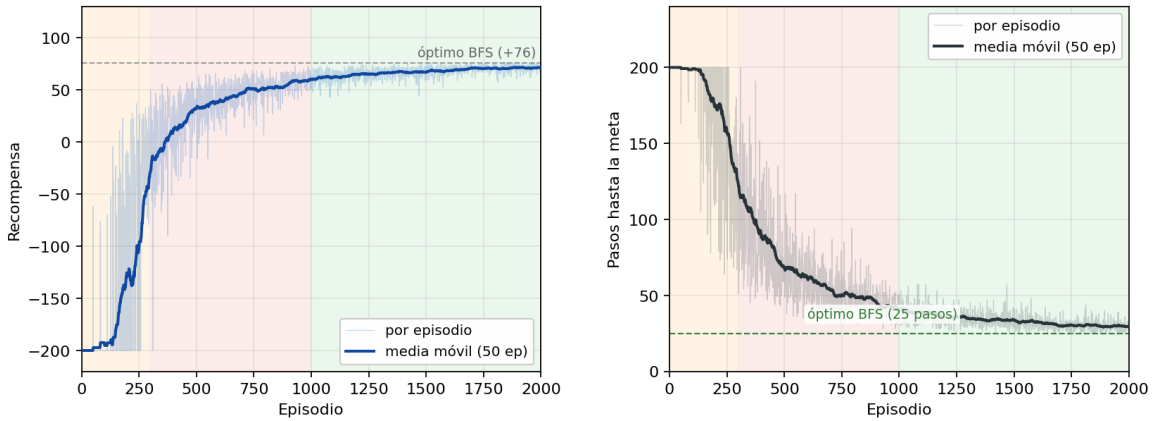


Figura 4: Curva de aprendizaje en el laberinto. Panel izquierdo: recompensa por episodio (línea fina cruda, línea gruesa media móvil de 50 episodios); la línea gris punteada en +76 marca el óptimo BFS. Panel derecho: pasos por episodio; la línea verde punteada en 25 es el óptimo BFS. El sombreado distingue las tres fases del entrenamiento: caos inicial (amarillo, ep 0–300), aprendizaje rápido (rosa, ep 300–1000) y convergencia (verde, ep 1000–2000).

Referencias

- [1] R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction*. 2.^a ed., MIT Press, 2018.
- [2] C. J. C. H. Watkins, P. Dayan. “Q-learning”. *Machine Learning*, vol. 8, n.º 3, 1992.
- [3] M. Chevalier-Boisvert, B. Dai, M. Towers, R. de Lazcano, L. Willems, S. Lahlou, S. Pal, P. S. Castro, J. Terry. “Minigrid & Miniworld: Modular & Customizable Reinforcement Learning Environments for Goal-Oriented Tasks”. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.